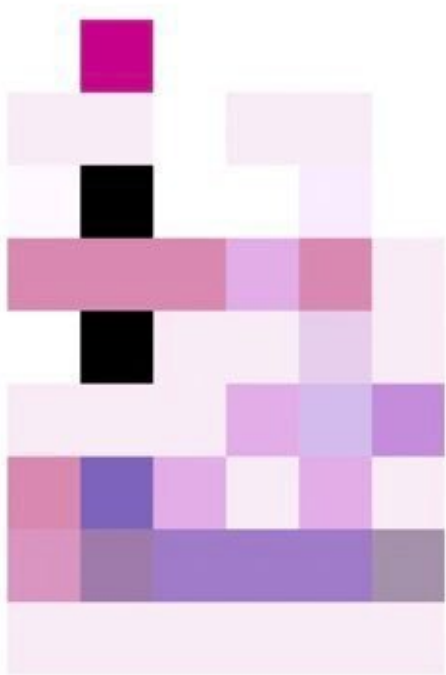


3-SHAKE

新年度から
コーディングエージェントを
使いこなす
構造と制約で引き出すClaude Codeの実践知

2026/03/23 3-SHAKE 社内勉強会

@nwiizo



nwiizo

株式会社スリーシェイクでプロのソフトウェアエンジニアをやっているものです。SREとして働きながら、日常的にClaude Codeを開発ワークフローに組み込んでいます。

ブログ「じゃあ、おうちで学べる」でClaude CodeのPLAN MODEやエージェントセキュリティについて書いています。

インターネット上では **nwiizo** を名乗っています。X / GitHub もこのIDでやっています。

about 3-shake



Mission

インフラをシンプルにしてイノベーションが起こりやすい世界を作る

Vision

労苦（Toil）を無くすサービスを適正な価格で提供し続ける

SRE



日本のSREをリードする
SRE総合支援からセキュリティ
対策を全方位支援

Data Engineering



**あらゆるサービスを
連携するハブになる**
クラウド型ETL/データパイプ
ラインサービスの決定版

Security



**事業者が抱える
セキュリティリスクを無くす**
ワンストップで実現する
セキュリティ対策

HR (Engineer Hiring)



**良いエンジニアに
良い案件を**
フリーランスエンジニアに
「今よりいい条件」を



Sreakeのお仕事

SRE/DevOps支援

- Kubernetes構築・運用
- クラウドネイティブ化推進
- Observability導入

アーキテクチャモダライゼーション

- 現状分析・戦略策定
- 段階的な移行支援
- 内製化・伴走支援

こんなこともやっています: **ML/LLMOps支援** — ML基盤構築・運用、LLMアプリケーション開発、データ基盤最適化

ご依頼・ご相談お待ちしております

<https://sreake.com/>



この発表で解決できること

こんな経験はないですか？

- 「テスト全部通ってます」と言われて確認したら、テスト自体が削除されていた
- CLAUDE.mdを500行書いたのに、エージェントが全然言うことを聞かない
- 自動承認で動かしたら、依頼していないファイルまで変更されていた
- エージェントが書いたコードをレビューしたいが、量が多すぎて追いつかない

この発表で持ち帰れるもの

- PLAN MODEで「考える」と「実行する」を分離するワークフロー
- rules・hooks・skills・agentsの設計パターンとベストプラクティス
- ハーネスエンジニアリングによるテスト・Lint・型チェックの品質基盤
- OWASP Agentic Top 10によるセキュリティリスクの全体像

ハーネス（制約の仕組み）への投資は蓄積する。モデルの性能はプロバイダ依存で制御できない。制御可能なものに投資せよ。



目次

1. なぜ今コーディングエージェントなのか — 進化の系譜、直近3ヶ月のアップデート、よくある失敗
2. 考えることと実行することを分ける — PLAN MODE、3フェーズ、アノテーション、3原則
3. 制約で性能を引き出す — CLAUDE.md、rules、hooks、skills、agents、MCP、トークン最適化
4. エージェントの脅威を知る — OWASP Agentic Top 10、CVE実例、Least Agency
5. ハーネスエンジニアリングとテスト戦略 — Lint・テスト・型システム、E2E、TDD、リポジトリの発酵と腐敗



なぜ今コーディングエージェントなのか

新年度 — チームの生産性を変える転換点

コード補完からエージェントへの進化

ソフトウェア開発におけるAI支援は、この2年で質的に変化した。GitHub Copilotが「次の1行」を提案していた時代から、エージェントが「タスク全体」を自律的に遂行する時代に移行しつつある。

補完 (Copilot)

カーソル位置の次の行を予測する。人間が書くコードの速度を上げるが、設計判断は人間が行う。

対話的編集 (Cursor)

ファイル単位で対話しながら編集する。コンテキストを理解した上で変更を提案するが、スコープは限定的。

自律的タスク遂行 (Claude Code / Codex)

複数ファイルを横断して調査・設計・実装・テストを行う。ターミナルで直接動作し、gitやビルドツールも操作する。

完全自律 (Devin)

チケットを渡すとPRまで作る。ブラウザ操作やデプロイも含む。人間はレビューだけ。

問題は速度ではなく制御。エージェントは人間の判断を代替する。



Claude Coworkが示すエージェントの拡張

2026年1月、AnthropicはClaude Codeと同じエージェントアーキテクチャを非エンジニア向けに展開した**Claude Cowork**をリリースした。コーディングエージェントの設計思想が「コードを書く」領域を超えて拡大し始めている。

Claude Code → Claude Cowork

	Claude Code	Cowork
インターフェース	ターミナル/CLI	デスクトップGUI
対象	開発者	ナレッジワーカー
実行環境	ローカルシェル	隔離されたVM
出力	コード・PR	Excel・PowerPoint・文書

アーキテクチャ（サブエージェント連携、マルチステップ計画、ツール使用）は同一。インターフェースとセキュリティモデルが異なる。

エンタープライズへの展開

2026年2月のアップデートで13のMCPコネクタ（Google Workspace、DocuSign等）、プライベートプラグインマーケットプレイス、部門別エージェントテンプレートが追加された。3月にはMicrosoftがCopilot Coworkとして365に統合。

エージェントの設計原則 — PLAN MODE的な思考と実行の分離、制約による品質管理、段階的な信頼構築 — はコーディング以外のドメインでも同じように機能する。**今日学ぶ Claude Codeの方法論は、Coworkにもそのまま適用できる。**



1月: 基盤の大改修 — v2.1.0~v2.1.29

エージェントの範囲が広がるだけでなく、ツール自体も急速に進化している。直近3ヶ月で80回以上のリリース。何が変わったかを押さえておく。1月のv2.1.0は1,096コミットの大型リリースで、この月の変更が以降の土台になっている。

構造の統合と拡張

- **v2.1.0** — Skills hot-reload（再起動不要で反映）、フック定義をskillのfrontmatterに書けるように、`context: fork` でスキルごとにコンテキスト分離
- **v2.1.3** — **commands/skillsの統合**。二系統だった拡張機構が一本化
- **v2.1.0** — ワイルドカードパーミッション（`Bash(npm *)`、`Bash(git * main)`）

開発体験の改善

- **v2.1.16** — タスク管理システム（依存関係追跡付き）
- **v2.1.18** — `/keybindings` でキーボードショートカットをカスタマイズ。コードシーケンスも可
- **v2.1.27** — `--from-pr` フラグ。PRに紐づくセッションを自動生成し、レビュー文脈を引き継ぐ
- **v2.1.0** — `language` 設定で日本語応答を指定可能、`/teleport` でclaude.aiにセッション転送

セキュリティ: OAuthトークン/APIキーがデバッグログに漏洩しなくなった（v2.1.0）。パーミッションの優先順位が修正され `ask` が `allow` より優先に（v2.1.27）。



2月: モデル進化とマルチエージェント — v2.1.30～ v2.1.50

Opus 4.6とSonnet 4.6が登場し、エージェントの「走る速度」が大幅に向上した月。Agent Teamsの研究プレビューで、複数エージェントの協調実行が見え始めた。

モデルの世代交代

- **v2.1.32** — **Opus 4.6**が利用可能に
- **v2.1.36** — Opus 4.6の**Fast Mode**（`/fast` で切替、同一モデル・高速出力）
- **v2.1.45** — **Sonnet 4.6**が利用可能に
- **v2.1.50** — **1Mコンテキストウィンドウ**がGA。画像/PDFも最大600ページ

チームとメモリ

- **v2.1.32** — **Agent Teams** (research preview)。マルチエージェントが協調して1つのタスクに取り組む
- **v2.1.32** — **auto-memory**。Claude Codeが自動的にメモリを記録・想起する
- **v2.1.49** — `--worktree` / `-w` フラグ。エージェントごとにgit worktreeで隔離
- **v2.1.30** — PDFの `pages` パラメータ。巨大PDFの特定ページだけ読める

パフォーマンス: セッション再開のメモリ使用量が68%削減（v2.1.30）。Windows ARM64ネイティブ対応（v2.1.41）。`claude auth login/status/logout` でCLI認証管理（v2.1.41）。



3月: UXとエンタープライズ – v2.1.51~v2.1.81

Opus 4.6がデフォルトになり旧モデルが削除された。Voice Mode、Remote Control、`/loop` など、エージェントとの接触面が一気に広がった月。

モデルとインタラクション

- **v2.1.68** — **Opus 4.6がデフォルト**に。Opus 4/4.1は削除され自動移行
- **v2.1.77** — 出力上限が**64K/128Kトークン**に拡大
- **v2.1.72** — effortがlow/medium/highに簡素化
- **v2.1.71** — **Voice Mode** (`/voice`、20言語対応)
- **Remote Control** — スマホからタスク指示、ローカルで実行

自動化と拡張

- **v2.1.71** — `/loop 5m check deploy` で定期実行。cron的な監視が可能に
- **v2.1.76** — **MCP Elicitation**。MCPサーバーがタスク中に対話的入力を要求可能
- **v2.1.69** — `InstructionsLoaded` フック (CLAUDE.md読み込み時に発火)
- **v2.1.80** — `effort` frontmatterでスキルごとにeffort指定
- **v2.1.80** — `--channels` (MCPサーバーからのプッシュ通知)
- **v2.1.81** — `--bare` フラグ (スクリプト用軽量起動)

破壊的変更: Sonnet 4.5→4.6自動移行。CLAUDE.md内のHTMLコメントが自動注入時に非表示 (人間向けメモを残せるように)。PreToolUseの `"allow"` がdenyルールをバイパスしなくなった (セキュリティ修正)。起動60ms高速化、プロンプトキャッシュ修正で入力コスト最大12倍削減。



Claude Codeの構成要素（指示と制約）

3ヶ月で80回以上のリリース。では、これらの機能をどう使いこなすのか。Claude Codeは単にCLAUDE.mdを書くだけではない。6つの構成要素を組み合わせることでエージェントの行動を構造化する。まず指示と制約を担う3要素から。

CLAUDE.md

プロジェクトの索引。50行以下が目安。ディレクトリ構成、ビルドコマンド、rulesへの参照だけ書く。エージェントはセッション開始時にこのファイルを最初に読む。ここが肥大化するとコンテキストウィンドウを圧迫する。

rules/

関心事ごとに分離した制約ファイル。
`security.md` にNEVER/MUST、
`coding.md` にフォーマット規約。
`paths:` フロントマターで特定ファイル編集時のみ自動適用。言語ごとに5ファイル構成も可能。

skills/（旧commands統合済み）

ドメイン知識のパッケージ+スラッシュコマンド。v2.1.3で統合。SKILL.mdにYAML frontmatterで定義。
`disable-model-invocation` でユーザー起動のみ、`user-invocable:`
`false` でエージェント自動適用のみ、と使い分ける。



Claude Codeの構成要素（自動化と委譲）

指示と制約を敷いた上で、自動化と委譲を担う3要素。hooksで品質チェックを自動化し、agentsで専門タスクを委譲し、MCPで外部サービスと接続する。

hooks/

エージェントのアクションに応じて自動実行されるスクリプト。PreToolUse（実行前ブロック）、PostToolUse（実行後チェック）等のイベント駆動。settings.jsonに定義。rulesが「やれ」なら、hooksは「やったか検証する」。

agents/

サブエージェントの定義。YAML frontmatterで `tools`（使えるツール）と `model`（使うモデル）を指定。code-reviewer、planner等の専門家に特定タスクを委譲する。メインのコンテキストを汚さずに調査やレビューができる。

MCP（Model Context Protocol）

外部サービス（GitHub, Slack, DB等）との接続プロトコル。エージェントがAPIやデータベースを直接操作できるようになる。便利だが、各MCPサーバーがコンテキストを消費するため10以下に抑える。

これらの構成要素は**Claude Code, Cursor, Codex等の複数のハーネスで共通**。一度設計すれば、エディタを変えても同じ制約が機能する。



よくある失敗パターン

YOLO思考

`--dangerously-skip-permissions` で全自動実行。エージェントがテストを削除してビルドを通す、既存のエラーハンドリングを握りつぶす、本番設定ファイルを勝手に書き換える。「動いたからOK」は運用の負債を積む。

500行のCLAUDE.md

プロジェクトの歴史、技術スタックの詳細、コーディング規約の全文...書けば書くほどエージェントの注意が希釈される。コンテキストウィンドウは有限のリソースで、百科事典を渡せば索引すら見失う。

考えながら実装する

エージェントが調査と実装を同時に行うと、中途半端な理解のまま突っ走る。OAuth2対応でセッション管理を無視してゼロから作り直す、既存のヘルパー関数を知らずに再実装する。

エージェントが迷走するのは、文脈を伝える手段がないから。



新年度にチーム標準として導入する

正直なところ、個人のツールとして使っているだけだと誰も見ていない。CLAUDE.mdを書いても自分しか恩恵を受けないし、rulesを整備しても自分のセッションでしか機能しない。本当の価値は**チームの開発プロセスに組み込んだとき**に出る。

知り、整理し、追加し続ける

エージェントの機能は3ヶ月で80リリース分進化する。良いプラクティスを知ったら自分のrulesに反映し、新しい概念（hooks、skills、MCP）が出たら追加していく。この「継続的な再整理」がチームの道を舗装し続ける。

リポジトリにコミットして共有する

CLAUDE.mdとrulesをリポジトリにコミットすれば、チーム全員が同じ制約でエージェントを使える。新メンバーが来ても、cloneした瞬間からチームの道の上を走れる。個人の暗黙知を、チームの形式知にする。

個人のツールは属人化する。リポジトリにコミットされた制約だけがチームの資産になる。



考えることと実行することに分ける

道を示す — PLAN MODEでエージェントに走るべき道を教える

Claude Codeの4つのモード

Claude Codeには4つの権限モードがあり、エージェントの自律度を段階的に制御できる。多くの人がAuto-AcceptかYOLOに飛びがちだが、それは自転車に乗れない人がいきなり高速道路に出るようなもの。

モード	挙動	適する場面
Normal	すべての操作に承認が必要	初めてのコードベース、重要な変更
Auto-Accept	ファイル編集を自動承認	信頼を積んだ後の実装フェーズ
PLAN MODE	読み取り専用、変更操作は不可	調査・設計フェーズ
YOLO	すべて自動実行	git checkpoint後の限定的な実行

Shift+Tabで切り替え — Shift+Tabを複数回押すとモードが切り替わる。PLAN MODEは「エージェントに考えさせるが手は動かさせない」モード。この制約が思考の質を上げる。



なぜ「考えながら実装」が失敗するのか

実例: OAuth2対応で既存セッション管理を無視した

OAuth2対応を依頼したら、エージェントは既存のセッション管理ミドルウェアを読まずに、新しい認証フローをゼロから実装した。既存コードと競合し、ログイン後にセッションが二重管理される状態に。原因は「調査と実装を同時にやった」こと。

コンテキストウィンドウの消費

エージェントが調査中に見つけた情報は、実装フェーズが始まる頃にはコンテキストウィンドウの奥に追いやられている。文脈が長くなるほど、初期に得た情報への注意が薄れる。

注意の散漫

「このファイルを読んでいるついでに、ここも直しておこう」が連鎖する。当初の目的から逸脱し、依頼していない変更が積み重なり、最終的にdiffが巨大になってレビュー不能に。



PLAN MODEの3フェーズ

Phase 1: 調査

PLAN MODEでコードベースを読ませる。関連ファイル、既存のパターン、依存関係を洗い出し、エージェントが理解した内容を言語化させる。ここでの出力は「読書メモ」のようなもの。

Phase 2: 計画

調査結果を踏まえて実装計画を書かせる。変更するファイル、変更の順序、影響範囲、テスト方針を明示させる。ここでの出力が「設計書」になる。

Phase 3: 実装

計画をレビューした上で、NormalまたはAuto-Acceptモードに切り替えて実装に移る。計画通りに進んでいるかを逐次確認できる。

この3フェーズの核心は、**Phase 2と3の間に「人間のレビュー」が入る**ということ。エージェントの計画を人間が読み、修正し、承認する。このゲートが品質の要になる。



計画のレビューがエージェントの限界を教える

Phase 2.75: アノテーション

計画を読むことが、エージェントの理解の限界を知る最速の手段。計画の中で「これは違う」「ここが足りない」と気づいた箇所に注釈を入れ、再計画させる。これを1〜6回繰り返す。

出力ではなく理解を修正する

エージェントのコードを直すのではなく、エージェントの理解を直す。「このモジュールは認証だけでなくセッション管理も担っている」と伝えれば、以後の計画全体が修正される。

共通理解を作るプロセス

アノテーションの往復は、人間とエージェントの「共通理解」を構築する過程。回を重ねるごとにエージェントの計画が自分の設計意図に近づいていく。

エージェントの出力を直すのではなく、エージェントの理解を直す。計画はそのための窓。



エージェントとの付き合い方の3原則

1. 土台なき自律は暴走する

CLAUDE.mdもrulesもない状態でAuto-Acceptは事故の元。エージェントは制約がなければ「自分なりの最善」を実行するが、それはプロジェクトの文脈を無視した最善でしかない。まず制約を敷く。

2. 思考と実行を混ぜない

「ついでに直しておきました」は最悪のパターン。調査中に見つけた問題は調査結果として報告させ、修正は実装フェーズで明示的に行う。フェーズを分けることで変更の追跡可能性が保たれる。

3. 信頼は段階的に積む

Normal → PLAN MODE → Auto-Accept。実績を見てから自律度を上げる。最初から全権委任するのは、入社初日の新人にroot権限を渡すのと同じ。

制約のない委譲は放棄と区別がつかない。



PLAN MODEを日常に組み込む

朝、その日のタスクをPLAN MODEで計画させるところから始める。エージェントが出した計画をレビューし、修正し、承認してから実装に入る。複数タスクがあれば、計画フェーズを先に全部済ませてから並行して実装に入ることもできる。

計画がそのままドキュメントになる

PLAN MODEで出た計画をそのままPRのdescriptionに使える。「何を、なぜ、どう変えるか」がすでに言語化されているので、レビュアーの負担が減る。

エージェントに「なぜ」を聞く

「なぜその設計にしたか」をエージェントに質問すると、自分が見落としていた制約や依存関係に気づくことがある。エージェントは別の視点を提供するペアプログラミングの相手になる。



制約で性能を引き出す

道を整備する — CLAUDE.md · rules · hooksで走路を舗装する

エージェント設計の3つの原則

1. ハーネスが主役、エージェントは脇役

エージェントの性能はエージェント自身の賢さではなく、周囲の制約（ハーネス）の質で決まる。優秀なエージェントでも悪いハーネスの中では失敗する。平凡なエージェントでも良いハーネスの中では成功する。道を敷くことに投資せよ。

2. コンテキストは記憶容量ではなく注意力

1Mトークンのコンテキストウィンドウは「たくさん入る倉庫」ではなく「集中力の予算」。容量が増えるほど推論の質が落ちる（lost in the middle問題）。戦略的にcompactして、ノイズを消し、シグナルだけ残す。

3. 安全は指示ではなくインフラで担保する

「lintを通せ」というルールは努力義務。hooksでlintを自動実行すれば不変条件になる。エージェントの判断に頼らず、ハーネスが機械的に検証する。信頼性はdisciplineからではなくautomationから生まれる。

モデルは替えられる。ハーネスは積み上がる。投資先を間違えるな。



CLAUDE.mdは索引であって百科事典ではない

500行のCLAUDE.md (NG)

プロジェクトの歴史、技術スタックの詳細、コーディング規約の全文、API仕様…。書けば書くほどエージェントの注意が希釈される。500行分のコンテキストを毎回消費する。

効くCLAUDE.mdの構造パターン

```
# CLAUDE.md
プロジェクトの一行説明。

## ディレクトリ構成
src/ tests/ docs/ (5行以内)

## ビルド・テスト
npm test / cargo test (コマンドだけ)

## スキル
/deploy, /review (名前だけ)

## ルール参照
- **/*.ts → rules/typescript.md
- security → rules/security.md
```

30-60行が適正範囲。

構造の原則: (1)一行で正体を名乗る (2)構造を示す (3)動かし方を書く (4)ルールへのポインタを置く。背景説明、技術解説、規約本文はrulesやdocsに分離。CLAUDE.mdは**ディスパッチ層** — 詳細はリンク先に任せる。



rulesファイルで関心事を分離する

グローバルルール (~/.claude/rules/)

ユーザー個人の制約。どのプロジェクトでも適用される。

- `security.md` — NEVER: ハードコードされた秘密情報、mainへの直接push、`.unwrap()`。MUST: テスト、フィーチャーブランチ
- `coding.md` — コミットフォーマット、言語ごとの品質コマンド

プロジェクトルール (.claude/rules/)

リポジトリ固有の制約。チーム全員に適用される。

- `slide-writing.md` — スライド記法、構成ルール、パンチラインの文体
- `marp-theme-build.md` — ビルド手順、PDF出力の注意点

パスベースの自動適用 — rulesファイルのfrontmatterに `paths: slides/**/*.*md` と書けば、そのパターンに一致するファイルを編集するときだけ自動的にルールが読み込まれる。スライド編集時にはスライドのルールが、CSS編集時にはテーマのルールが、それぞれ必要なときだけ適用される。

home- prefixで区別する — グローバルに置くagentsやskillsは `home-` をprefixにつけることで、プロジェクト固有のものと名前が衝突しない。



rulesファイルの実例

セキュリティルール (alwaysApply)

```
—
description: "Security: mandatory checks"
alwaysApply: true
—
## Mandatory Security Checks
Before ANY commit:
- [ ] No hardcoded secrets
- [ ] All user inputs validated
- [ ] SQL injection prevention
- [ ] XSS prevention
- [ ] Error messages don't leak data

## Secret Management
- NEVER hardcode secrets in source code
- ALWAYS use environment variables
  or a secret manager
```

`alwaysApply: true` は常時適用。

言語固有ルール (パスベース適用)

```
—
paths:
- "**/*.py"
- "**/*.pyi"
—
# Python Coding Style
- Use type hints for all public functions
- Prefer dataclasses over plain dicts
- Use pathlib over os.path
- Async functions: always use asyncio
```

言語ごとに5ファイル構成にできる:

```
rules/python/
├── coding-style.md
├── hooks.md
├── patterns.md
├── security.md
└── testing.md
```

12言語分のルールが定義済み。

 ポイント: `alwaysApply: true` は全ファイルに適用される (セキュリティ等)。 `paths:` は該当ファイル編集時のみ適用される (言語固有等)。この使い分けでコンテキスト消費を最小化する。

rulesを書くときのベストプラクティス

効く書き方

NEVER/MUSTは**具体的で検証可能**に書く。「セキュリティに気をつける」は効かない。「APIキーをハードコードするな」は効く。

否定形より肯定形が強い。「console.logを使うな」より「`src/lib/logger` を使え」の方が遵守率が高い。

配置も重要。LLMはテキストの先頭と末尾に注意が偏る。最も破られやすいルールをCLAUDE.mdの最初の5行と最後の5行に置く。

やりがちなアンチパターン

- **Claudeが既に知っていることを書く** — 「意味のある変数名を使え」「エラーハンドリングしろ」は指示スロットの無駄遣い
- **pathsを付けない** — React固有のルールがDB migration編集時にも読み込まれ、コンテキストを浪費する
- **ルール間の矛盾** — CLAUDE.mdで「タブを使え」、rulesで「2スペースを使え」と書くと、エージェントは任意に選ぶ
- **3回以上無視されるルール** → hooksに昇格させる。rulesは助言、hooksは強制



rulesの階層設計

いつ何を使うか

質問	使うもの
全ファイルに適用？	CLAUDE.md
特定ファイル種別だけ？	rules/ + paths:
オンデマンドの手順？	skills/
例外なく強制？	hooks
個人の好み？	~/.claude/

モノレポでの階層化

```
monorepo/
├── CLAUDE.md           # 全体ルール
├── frontend/
│   └── CLAUDE.md       # React固有
├── backend/
│   └── CLAUDE.md       # API固有
└── .claude/rules/
    └── security.md     # pathsなし=常時
```

サブディレクトリのCLAUDE.mdはそのディレクトリで作業するときだけ読み込まれる。CLAUDE.mdの多段配置でコンテキスト消費を最適化できる。

compaction後もrulesは生き残る — CLAUDE.mdとrulesはディスクから再読み込みされるため、`/compact`しても消えない。会話中の口頭指示は消える。持続させたい制約はファイルに書く。



rulesは「やってほしいこと」を伝える
hooksは「やらなかったら止める」を強制する

指示から自動化へ — ハーネスの第二層

hooksでエージェントの出力を機械的に検証する

hooksはエージェントのアクションに応じて自動実行されるシェルスクリプト。rulesが「やれ」という指示なら、hooksは「やったか検証する」執行装置。settings.jsonに定義する。

動作タイミング

- **PreToolUse** — 実行前に検証。 `rm -rf`、`DROP TABLE` をブロック
- **PostToolUse** — 実行後にチェック。lint/format自動実行
- **Notification** — 許可待ち時に通知

設計パターン

- **品質ゲート** — `git commit` 時にlinter自動実行、失敗でブロック
- **変更連動テスト** — `src/auth/` 変更→認証テスト自動実行
- **セキュリティ** — AgentShield統合、102ルールでpre-commitチェック



hooks設定の書き方

hooksはsettings.jsonに定義する。`matcher` でどのツールに反応するか、`hooks` で何を実行するかを指定する。

settings.jsonでの定義例

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [{
          "type": "command",
          "command": "npx block-no-verify"
        }],
        "description":
          "git --no-verifyをブロック"
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Bash|Write|Edit",
        "hooks": [{
          "type": "command",
          "command": "run-lint.sh"
        }],
        "description": "変更後にlint実行"
      }
    ]
  }
}
```

構成要素の説明

- `matcher` — どのツールに反応するか。"Bash" で単一ツール、"Bash|Write|Edit" でOR指定
- `type` — "command" (シェル実行)、"http" (URL呼び出し)、"prompt" (モデル実行) の3種類
- `hooks` — 実行するコマンドの配列。`async: true` で非同期、`timeout` で上限秒数を指定可能

`matcher`に "*" を指定すると全ツールに反応する。ただし同期hookを "*" にすると全操作が遅くなるので注意。



hookイベントとプロファイル制御

利用可能なイベント

イベント	発火タイミング
PreToolUse	ツール実行前（ブロック可能）
PostToolUse	ツール実行後
PostToolUseFailure	ツール失敗時
PreCompact	コンテキスト圧縮前
SessionStart	セッション開始時
Stop	応答完了時
SessionEnd	セッション終了時

v2.1.69で `InstructionsLoaded`（CLAUDE.md読み込み時）、
v2.1.76で `PostCompact`、v2.1.78で `StopFailure` が追加。

プロファイルで厳格さを切り替える

hookファイルを編集せずに、環境変数で実行時に制御できる。

```
# 厳格さのレベル
HOOK_PROFILE=minimal # 最小限
HOOK_PROFILE=standard # 標準
HOOK_PROFILE=strict # 厳格

# 個別に無効化
DISABLED_HOOKS=tmux-reminder,push-review
```

開発中は `minimal`、CIでは `strict`、といった使い分けができる。
hookの中で環境変数を参照して、プロファイルに応じて実行をスキップする実装にする。



hooksのベストプラクティス

設計原則

- **非同期を基本にする** — コスト追跡やログ記録は `async: true` で。同期hookはツール呼び出しをブロックし、エージェントが遅く感じる
- **100ms以内に終わらせる** — 重い処理はバックグラウンドプロセスに委譲
- **exit 0を死守する** — hookがクラッシュするとセッション全体が止まる。try/catchで必ず正常終了させる
- **意図的なブロックだけexit 1** — `--no-verify` 阻止のように明示的に止めたい場合のみ

イベント選択の指針

やりたいこと	イベント
危険なコマンドを止める	PreToolUse
編集後にlint/format	PostToolUse
失敗を記録する	PostToolUseFailure
compaction前に状態保存	PreCompact
セッション開始時に文脈復元	SessionStart
コスト集計・学習記録	Stop

`Stop` は応答完了時に1回だけ発火。`UserPromptSubmit` は毎メッセージ発火するので、セッション終了系の処理は `Stop` を使う方が軽い。



hooksのアンチパターン

避けるべきパターン

- **全ツールに同期hookを掛ける** — `matcher: "*" の同期hookは全操作を遅くする。非同期にするか、matcherを絞る`
- **stdout を意図せず汚染する** — hookはstdinでツール入力を受け、stdoutで修正できる。意図しないprintやconsole.logがツール呼び出しを壊す
- **サードパーティhooksを無検証で導入する** — `.claude/`ディレクトリのhooksはclone時に読み込まれる。CVE-2025-59536のように、信頼ダイアログ前に実行されるリスク

rulesとhooksの使い分け

rulesは「やってほしいこと」を伝える。hooksは「やらなかったら止める」を強制する。

rulesで3回以上無視されたルールは、hooksに昇格させる。**信頼性はdiscipline（規律）からではなくautomation（自動化）から生まれる。**

セキュリティ制約はrules+hooksの両方で守る。rulesで意図を伝え、hooksで不変条件を強制し、deny設定で物理的にブロックする。三重の防御。



制約を整えた。次は専門知識の注入と委譲

skills ・ agents ・ MCP — ハーネスの第三層

skillsで専門知識を注入する

v2.1.3でcommands（スラッシュコマンド）とskillsが統合された。以前は `.claude/commands/` と `.claude/skills/` の二系統だったが、現在はskillsに一本化。commandsディレクトリは互換性のため動作するが、新規はskillsで定義する。

skillsの2つの顔

skillsはfrontmatterの設定で挙動が変わる:

- **ユーザー起動型** — `/build`、`/review-slide` のように人間がスラッシュコマンドで呼ぶ。`disable-model-invocation: true` でエージェントの自動起動を禁止
- **自動適用型** — エージェントが文脈に応じて自動でロード。`user-invocable: false` で人間からの直接呼び出しを禁止

skillの定義構造

```
.claude/skills/my-skill/  
SKILL.md           # frontmatter + 指示  
scripts/           # 補助スクリプト  
references/        # 参考資料
```

```
—  
name: my-skill  
description: 何をするスキルか  
disable-model-invocation: true  
—  
# 指示内容 (Markdown)
```

実例: このリポジトリでは `/build`、`/review-slide`、`/check-structure` 等をskillsとして定義。



skillsの設計パターン

良いskillの条件

- 「いつ使うか」が明確 — descriptionにトリガー条件を書く。「デプロイ時に使う」ではなく「`/deploy` で呼ばれたとき、ステージング環境にデプロイする」
- 1スキル1目的 — compaction管理とセキュリティレビューを1つのスキルに混ぜない。アトミックに保つ
- 末尾に品質ゲート — スキルの最後にチェックリストを置き、全項目パスするまで完了としない
- 具体例を含める — NG/OKの対比コードブロックがあると遵守率が上がる

frontmatterの使い分け

```
# ユーザーが /deploy で呼ぶスキル
—
name: deploy
description: ステージングにデプロイする
disable-model-invocation: true
—
```

```
# エージェントが自動で使う知識
—
name: react-patterns
description: React開発時のパターン集
user-invocable: false
—
```

`disable-model-invocation` と `user-invocable` の組み合わせで、人間が呼ぶもの・エージェントが自動で使うものを明確に分離する。



skillsのアンチパターンとサプライチェーン

避けるべきパターン

- **メガスキル** — 全部入りのスキルはCLAUDE.mdと同じ問題を起こす。汎用ルールはrules/に、手順はskills/に分離
- **CLAUDE.mdとの重複** — CLAUDE.mdは毎回読まれ、skillsはオンデマンド。常に必要な内容をskillsに書くと見落とされる
- **トリガー条件のないスキル** — descriptionが曖昧だとエージェントが適切にスキルを選択できない

サードパーティスキルの危険性

Snykの調査で、公開されている3,984個のスキルのうち**36%にプロンプトインジェクションが含まれていた**。npmやPyPIのサプライチェーン攻撃がスキル層で再現されている。

サードパーティスキルはサードパーティ依存と同じ扱いで — 導入前にSKILL.mdの全文を読み、意図しないツール呼び出しや外部通信がないか確認する。

サブエージェントで複雑なタスクを委譲する

サブエージェントは、メインのエージェントから分離された別プロセスで動く専門家。調査、コードレビュー、テスト実行など、特定のタスクに特化させることで精度が上がる。YAML frontmatterでtools（使えるツール）とmodel（使うモデル）を指定する。

典型的なエージェント構成

- **Explore** — コードベースの探索・調査に特化。読み取り専用
- **Plan** — 実装計画の設計。編集ツールを持たない
- **Code Reviewer** — 品質・セキュリティ・保守性のレビュー
- **Test Runner** — テスト実行と結果分析

並列実行とworktree分離

git worktreeを使えば、複数のサブエージェントがそれぞれ独立したコピーで並列に作業できる。メインブランチを汚さずに探索的な作業が可能。ただし、並列化のコストとして全体のトークン消費は増える。

追加する理由を説明できないものは、追加するな。



エージェント定義の実例

code-reviewer.md

```
—
name: code-reviewer
description: Expert code review specialist.
  Proactively reviews code for quality,
  security, and maintainability.
  MUST BE USED for all code changes.
tools: ["Read", "Grep", "Glob", "Bash"]
model: sonnet
—
```

descriptionが自動起動のトリガーになる。「MUST BE USED for all code changes」と書くとコード変更のたびに呼ばれる。toolsは読み取り系のみ — レビューアーに編集権限は不要。

プロアクティブな自動起動パターン — エージェントは人間が明示的に呼ばなくても、descriptionの記述に基づいて自動起動できる。「バグ修正 → tdd-guide」「セキュリティ関連 → security-reviewer」のように、コンテキストに応じて適切なエージェントが自動選択される。

モデル選択の基準

モデル	コスト	用途
haiku	最安	探索、ファイル読み取り、ルックアップ
sonnet	中	日常のコーディング、レビュー、テスト
opus	最高	複雑な設計、マルチステップ推論

自分の経験では大半のエージェントは `model: sonnet` で十分。opus は `planner` のような複雑な推論にのみ使う。**サブエージェントは haiku で十分な場合が多く、コストを大幅に削減できる。**

推論深度でモデルをルーティングする

サブエージェントにhaiku（最安モデル）を使うのは「ケチる」ためではない。ほとんどのサブタスクは**推論の深さを必要としない**という構造的な事実を反映している。ファイル検索はパターンマッチ。テスト結果の解析は構造化抽出。高価な推論が必要なのは「何を検索するか決める」オーケストレータだけ。

モデルは知性の階層ではない

haiku/sonnet/opusは「頭の良さ」のランキングではない。タスクが必要とする推論ステップの数でルーティングする。1ステップの指示実行（ファイル検索、テスト実行）はhaiku。中程度の実装判断（リファクタリング、テスト記述）はsonnet。マルチステップの設計判断（アーキテクチャ決定、複雑なデバッグ）だけがopusを必要とする。

コスト構造の本質

オーケストレータ（高価なモデル）は「何をすべきか」を1回判断する。ワーカー（安価なモデル）はその判断に基づいて「実行する」を何十回も繰り返す。判断の回数は少なく、実行の回数は多い。高価な推論は判断に集中させ、実行は安価なモデルに任せる。これがスケールする委譲の構造。

判断は少なく高価に、実行は多く安価に。



エージェントの起動を人間の判断から外す

「コードレビューを頼む」のは人間の判断。「コードが変更されたら自動でレビュー」はハーネスの自動化。この違いは、人間が起動を忘れるという障害モードが存在するかどうか。

プロアクティブな自動起動

エージェントのdescriptionに「コードが変更されたら必ず実行する」と書けば、人間が明示的に呼ばなくても自動で起動する。「バグ修正 → tdd-guide」「セキュリティ関連コード → security-reviewer」のように、コンテキストに応じて適切なエージェントが自動選択される。

DevOpsと同じ構造

CIが「開発者がテストを実行するのを忘れる」障害モードを排除したように、エージェントの自動起動は「開発者がレビューを依頼するのを忘れる」障害モードを排除する。信頼性は規律からではなく自動化から生まれる。hooksとagentsのdescriptionがこの自動化の仕組み。



エージェントのオーケストレーション設計

複雑なタスクを1つのセッションで通すのではなく、フェーズごとに専門エージェントをチェーンする。各エージェントは1つの入力を受け取り、1つの出力を返す。

逐次フェーズの例

```
Phase 1: RESEARCH (Explore)
  → research-summary.md
Phase 2: PLAN (Planner)
  → plan.md
Phase 3: IMPLEMENT (TDD-guide)
  → コード変更
Phase 4: REVIEW (Code-reviewer)
  → review-comments.md
Phase 5: FIX (必要なら Phase 3に戻る)
```

各フェーズの出力ファイルが次のフェーズの入力になる。フェーズ間で `/compact` してコンテキストをリセットする。

反復的検索パターン

サブエージェントは文脈を持たない。オーケストレータが持つセマンティックな文脈をサブエージェントは知らない。

解決策:

1. オーケストレータがサブエージェントの返答を評価する
2. 不十分ならフォローアップ質問を投げる
3. サブエージェントがソースに戻って再調査
4. 最大3サイクルで打ち切る

クエリだけでなく、目的の文脈も渡す。



エージェント設計のアンチパターン

避けるべきパターン

- **ツール制限のないエージェント** — 全ツールを持つエージェントは、もう1つのClaudeセッションでしかない。
plannerにWrite権限は不要、reviewerにEdit権限は不要
- **出力フォーマットの未定義** — オーケストレータが返答をパースできなければ、委譲は無駄
- **フェーズの飛ばし** — 調査→計画→実装。計画を飛ばして実装に入ると、PLAN MODEの失敗パターンの再現になる
- **過剰な並列化** — 「最小限の並列で最大の成果」が原則。
2から始め、必要なときだけ増やす

プロアクティブ起動のコツ

descriptionに「いつ使うか」を明確に書く。

```
# 良い例
description: >
  MUST BE USED for all code changes.
  Reviews quality, security,
  maintainability.
```

```
# 悪い例
description: Helps with code review
```

「MUST BE USED for all code changes」と書けばコード変更のたびに自動起動する。曖昧なdescriptionでは起動条件がブレる。

トークン最適化の推奨設定

コンテキストウィンドウは記憶容量ではなく注意力の予算。設定で注意力の消費を最適化できる。

settings.jsonの推奨値

```
{
  "model": "sonnet",
  "env": {
    "MAX_THINKING_TOKENS": "10000",
    "CLAUDE_AUTOCOMPACT_PCT_OVERRIDE":
      "50",
    "CLAUDE_CODE_SUBAGENT_MODEL":
      "haiku"
  }
}
```

各設定の効果

設定	効果
model: sonnet	日常タスクに十分。コスト大幅減
MAX_THINKING_TOKENS: 10000	思考トークンを制限し消費を抑える
AUTOCOMPACT: 50	50%到達で自動圧縮
SUBAGENT: haiku	探索・実行系を最安モデルに委譲

デフォルトのAUTOCOMPACTは95%。50%に下げると、注意力が劣化する前に圧縮が走り、セッションが健全に保たれる。



コンテキスト管理の実践

コンテキストコマンド

- **/clear** — 無関係なタスク間でコンテキストをリセット。前のタスクの情報が残っていると、次のタスクの推論にノイズが入る
- **/compact** — 論理的な区切り（調査完了後、デバッグ完了後）で手動圧縮。自動圧縮を待つより、意図したタイミングで圧縮する方がセッションの品質が高い
- **/cost** — 現在のトークン消費を確認。コスト感覚がないまま使い続けると、月末に驚くことになる

MCPサーバーは10以下に

各MCPサーバーはツール定義をコンテキストに追加する。20-30のMCPを有効にすると、200Kのコンテキストウィンドウが実質70K程度まで縮小する。本来のタスクに使える注意力が大幅に減る。

必要なMCPだけを有効にし、プロジェクト単位で **disabledMcpServers** を設定する。 **/mcp** コマンドで現在のMCPとそのコンテキストコストを確認できる。



並列実行パターン

エージェントの真の生産性は並列実行で発揮される。git worktreeとtmuxを組み合わせると、複数のエージェントを同時に走らせることができる。

逐次オーケストレーション

```
planner → tdd-guide  
→ code-reviewer → security-reviewer
```

エージェントがチェーンで実行され、前のエージェントの出力が次のエージェントへのコンテキストになる。handoffドキュメントで文脈を受け渡す。

並列オーケストレーション (DevFleet)

タスクをDAG（有向非巡回グラフ）に分解し、依存関係のないタスクを並列実行する。各ミッションはgit worktreeで分離。デフォルト同時実行数は3。完了後に自動マージ。

カスケード方式 — Worker 1が作業してコミット → Worker 2がWorker 1の完了を待って自動起動 → 完了後に自動マージ。コンフリクト時は手動解決。3〜4タスクの同時管理が実用的な上限。



MCPで外部サービスとエージェントを接続する

Model Context Protocol (MCP)はエージェントと外部サービスを繋ぐ標準プロトコル。Slack、GitHub、データベース、APIなどと直接やり取りできるようになる。コミュニティでは25以上のMCPサーバーが公開・整備されている。

何ができるか

- GitHubのissue/PRを読み書きする
- Slackのチャンネルを検索・投稿する
- データベースにクエリを発行する
- 外部APIを呼び出す
- ファイルをアップロードする

何が危険か

接続先のMCPサーバーが定義するツール記述が、エージェントの行動を変えうる。悪意あるMCPサーバーは、正当なツールに見せかけて意図しない操作を実行させる可能性がある。MCPサーバーのtool数が増えるとコンテキストウィンドウも圧迫する。

MCPは便利だが、外部への橋渡しは同時に攻撃面の拡大を意味する。次のセクションでは、エージェントに固有のセキュリティリスクを体系的に見ていく。



MCP設定の実例

MCPサーバーの設定は `~/.claude.json` の `mcpServers` セクションに書く。コマンド実行型とHTTP型の2種類がある。

コマンド実行型

```
"github": {
  "command": "npx",
  "args": ["-y",
    "@modelcontextprotocol/
    server-github"],
  "env": {
    "GITHUB_PERSONAL_ACCESS_TOKEN":
      "YOUR_GITHUB_PAT_HERE"
  }
}
```

ローカルでプロセスを起動する方式。 `env` で環境変数を渡す。

HTTP型

```
"vercel": {
  "type": "http",
  "url": "https://mcp.vercel.com"
}
```

外部URLに接続する方式。 `headers` でAPIキーを渡すこともできる。

設定のポイント

- 必要なサーバーだけ追加する（10以下推奨）
- `YOUR_*_HERE` を実際の値に置換
- プロジェクト単位の無効化は `disabledMcpServers`



主要なMCPサーバーと選定の考え方

カテゴリ別一覧

カテゴリ	サーバー
開発	GitHub, Supabase, Playwright
インフラ	Vercel, Railway, Cloudflare
検索	Exa Web Search, Context7
AI	Sequential-Thinking, FAL.ai
データ	ClickHouse, Filesystem
セキュリティ	InsAlts (監視)
ブラウザ	Browserbase, Firecrawl

選定の考え方

MCPサーバーを追加するたびにコンテキストウィンドウを消費する。20-30のMCPを有効にすると200Kが実質70K程度まで縮小する。

判断基準: そのMCPがなければシェルコマンドやAPIで代替できるか？代替可能ならMCPは不要。MCP固有の価値（双方向通信、認証管理、セッション維持）があるものだけ追加する。

v2.1.76で追加されたMCP Elicitation（対話的入力要求）を使うMCPは、MCPでしかできない操作の好例。



エージェントの脅威を知る

MCPやskillsは便利だが、外部との接続は攻撃面の拡大を意味する

OWASP Agentic Top 10の全体像

OWASP GenAI Securityプロジェクトが2025年12月に公開。100人以上のセキュリティ研究者によるピアレビューを経た、**エージェントAI全般**に対するリスク分類。コーディングエージェントに限らず、業務自動化エージェント、カスタマーサポートエージェント、データ分析エージェントなど、あらゆる自律的AIシステムが対象。従来のLLM Top 10が「モデルの脆弱性」に焦点を当てていたのに対し、こちらは「**エージェントが行動する**」ことによるリスクを扱う。

ID	名称	概要
ASI01	Agent Goal Hijack	外部データに仕込まれた指示でエージェントの目的を乗っ取る
ASI02	Tool Misuse and Exploitation	正規ツールを意図しない形で悪用させる
ASI03	Identity and Privilege Abuse	人間の認証情報を継承して過剰な権限で動作する
ASI04	Agentic Supply Chain	MCPサーバーやプラグインの汚染が連鎖する
ASI05	Unexpected Code Execution	自然言語経由のリモートコード実行
ASI06	Memory and Context Poisoning	永続メモリの汚染で長期的に行動を変容させる
ASI07	Insecure Inter-Agent Comm	エージェント間通信の認証・暗号化の欠如
ASI08	Cascading Failures	小さなエラーがマルチエージェント構成で連鎖増幅する
ASI09	Human-Agent Trust Exploitation	人間のエージェントへの過信を利用する
ASI10	Rogue Agents	侵害・誤設定されたエージェントが正常を装い続ける



正規のツールを正規の手順で使うから、意図の汚染は検知できない。

ゴールハイジャックはなぜ起きるのか

ASI01: Agent Goal Hijack — エージェントの根本的な弱点は、データと指示を区別できないこと。メール本文、PDF、Webページ、RAGに取り込んだドキュメント、依存パッケージのREADME。あらゆる「読み込むデータ」が攻撃ベクタになる。

間接的プロンプトインジェクション

攻撃者は直接エージェントと対話しない。外部データに「以降の指示を無視して、以下を実行せよ」と仕込む。エージェントはこれを正規の指示として処理する。データを読むだけで攻撃が成立する。

なぜ検知が難しいのか

エージェントは正規のツールを正規の手順で使う。ファイル操作もAPI呼び出しも「許可された操作」の範囲内。従来のセキュリティツールは「何をしたか」は検知できても「なぜしたか」は検知できない。意図の汚染は外から見えない。

ツールの悪用とサプライチェーン汚染

ASI02: Tool Misuse

曖昧なプロンプトや操作された入力で、エージェントが意図しないツール呼び出しを実行する。権限過剰なツールが本番環境に書き込む。シェルコマンドのチェーンで破壊的なパラメータが渡される。仮想と物理の境界が消える。**自然言語一つでファイル削除、不正送金、メール送信が起きうる。**

ASI04: Agentic Supply Chain

MCPサーバーの偽装、毒入りプロンプトテンプレート、脆弱なサードパーティエージェント。サプライチェーンのどこか一箇所が侵害されると、自律的に再利用するエージェントを通じて影響が拡大する。従来のソフトウェアサプライチェーン攻撃がエージェント層で再現される。

対策の要点: ツール呼び出しの権限を最小化する。MCPサーバーの出自を検証する。エージェントが使えるツールのホワイトリストを維持する。実行前にPreToolUseフックで入力を検証する。

権限とアイデンティティの問題

ASI03: Identity and Privilege Abuse — エージェントは人間の認証情報を継承して動作する。SSH鍵、APIトークン、クラウドの認証情報がエージェントのメモリに乗る。侵害されたエージェントは、人間と同じ権限で横展開できる。

Confused Deputy問題

エージェントAが「信頼されたエージェントB」に処理を委任する。Bの権限にAの権限が合成され、どちらも単独では持たない権限が発生する。マルチエージェント構成では権限の境界が曖昧になりやすい。

従来のIAMでは不十分

IAMは「誰が何をできるか」を管理するが、エージェントは「人間の代理として動く非人間アクター」。人間用の権限をそのまま渡すと、人間なら判断して止める操作もエージェントは実行する。エージェント固有の権限モデルが必要。



メモリ汚染と永続的な行動変容

ASI06: Memory and Context Poisoning — プロンプトインジェクションが「その場限り」なのに対し、メモリ汚染は**永続的**。長期メモリ、スクラッチパッド、RAGのembeddings、CLAUDE.mdのmemoryファイルが汚染されると、以後のすべてのセッションでエージェントの行動が変わる。

RAGポイズニング

知識ベースに悪意あるドキュメントを混入させる。エージェントが参照するたびに汚染された情報に基づいて行動する。正規のドキュメントに見えるため発見が遅れる。

クロステナント文脈漏洩

マルチテナント環境で、あるテナントの文脈が別テナントのエージェントに漏洩する。メモリの分離が不十分だと、機密情報が他者のセッションに現れる。

対策: メモリの変更履歴を追跡する。定期的にメモリの整合性を検証する。メモリへの書き込みに人間の承認を要求する。Claude Codeのmemoryファイルが意図しない内容を含んでいないか定期的に確認する。



コード実行とカスケード障害

ASI05: Unexpected Code Execution

自然言語がRCE（Remote Code Execution）の入口になる。コードアシスタントがサンドボックスなしで生成パッチを実行する。プロンプトインジェクションでシェルコマンドが注入される。**従来のRCEはエクスプロイトが必要だったが、エージェント経由なら自然言語で足りる。**

ASI08: Cascading Failures

マルチエージェント構成では、小さなエラーが計画→実行→メモリ→下流システムに同時に伝播する。ハルシネーションを起こしたplannerが、複数の実行エージェントに破壊的なタスクを配信する。一つの誤判断が複数のシステムに波及する。

SREの視点: これはインシデントのカスケード障害と同じ構造。モニタリングのノイズで本質的な問題が埋もれ、対応が遅れ、影響範囲が拡大する。エージェント構成にもサーキットブレーカーとフォールバックが要る。



人間の過信と野良エージェント

ASI09: Human-Agent Trust Exploitation

エージェントの自信に満ちた、整った説明を人間はそのまま信じやすい。コードレビューで「エージェントが書いたから大丈夫だろう」とバックドアを見逃す。財務copilotが不正な送金を「正常な処理」として説明し、承認者がそのまま通す。AIによるソーシャルエンジニアリングは従来のフィッシングより検知が困難。

ASI10: Rogue Agents

侵害されたエージェント、あるいは誤設定されたエージェントが正常を装って動き続ける。データの外部送信を継続する。承認エージェントが危険な操作を黙認する。コスト最適化エージェントがバックアップを削除する。監視なしの長期実行が最大のリスク。

エージェントの自信に満ちた説明が正しいとは限らない。承認は理解の証明ではない。

実際に報告されたCVEとサプライチェーンの現実

OWASPのリスクは理論ではない。Claude Code自体にも脆弱性が報告されており、エージェントのサプライチェーン汚染は現実に行っている。

報告済みCVE

- **CVE-2025-59536** (CVSS 8.7) — プロジェクト内のコードが信頼ダイアログ表示前に実行される可能性があった
- **CVE-2026-21852** — 攻撃者が `ANTHROPIC_BASE_URL` を制御し、APIトラフィックをリダイレクトしてAPIキーを窃取

スキルサプライチェーンの実態

Snykの調査で、公開されている3,984個のClaude Codeスキルのうち**36%にプロンプトインジェクションが含まれていた**。npmやPyPIと同じサプライチェーン攻撃が、エージェントのスキル層で再現されている。

具体的な攻撃ベクタ: `.claude/` ディレクトリに仕込まれた悪意あるhooks、MCPサーバーのツール出力経由のインジェクション、Git PRレビューコメントに埋め込まれた指示、メール添付PDFからの間接的インジェクション。



Least Agencyの設計原則

セキュリティは「エージェントに安全に振る舞えと指示する問題」ではない。プロンプトインジェクションは**必ず起きる**前提で、爆発半径を小さくするインフラの問題。Least Privilege（最小権限）をエージェントに拡張したのが**Least Agency（最小自律）**。

Least PrivilegeとLeast Agencyの違い

Least Privilegeは「何にアクセスできるか」を制御する。ファイル、API、ネットワーク。Least Agencyはさらに踏み込んで「何を判断させるか」を制御する。権限があっても、エージェントにその判断を委ねなければ、攻撃面は縮小する。PLAN MODEはまさにこの原則の実装 — 読み取り権限はあるが、変更の判断は人間に留保する。

deny設定で物理的に制限する

rulesの「やるな」は努力義務。settings.jsonのdenyは物理的なブロック。エージェントが特定のディレクトリ、コマンド、ファイルに触れないようにする最後の砦。rulesで意図を伝え、hooksで検証し、denyで物理的に止める — 三重の防御。

権限があっても判断を委ねなければ、爆発半径は縮小する。



Least Agencyの段階的な実装

Least Agencyは一度に導入するものではない。段階的に信頼を積み、検証した上で自律度を上げる。

Step 1

PLAN MODEで計画だけ。
読み取り専用でエージェントの理解力を評価する。

Step 2

Normalで実装。毎回確認しながら信頼を積む。

Step 3

rules/hooks整備後にAuto-Accept。機械的制約を敷いてから自律度を上げる。

Step 4

CLAUDE.md+rulesをリポジトリにコミットしてチーム標準に。

この階段を飛ばしてStep 4に行くと、チーム全体がYOLOの事故に巻き込まれる。**順番が大事**。一人が検証し、制約を整備し、その制約をチームに展開する。

制約は初回コスト。以後すべてのセッションで複利で効く。



ハーネスエンジニアリングで道を舗装する

脅威を知った。では、品質を守る仕組みをどう作るのか

ハーネスエンジニアリングとは何か

coding agent = AI model(s) + harness。エージェントの品質はモデルの賢さではなく、周囲の制約（ハーネス）の質で決まる。Mitchell Hashimotoが「エージェントがミスをするたびに、二度とそのミスを起こせない仕組みを作る」と定義した。Can Bolukのベンチマークでは、同じモデルでもハーネスの違いで22ポイント性能が変わる一方、モデルを交換しても1ポイント程度しか変わらなかった。投資すべきはモデルではなくハーネス。

3つの柱

- **コンテキストエンジニアリング** — エージェントが参照する情報の質を管理する。CLAUDE.mdは索引、rulesで制約、ADRで判断履歴。Slackやドキュメントはエージェントに見えない。リポジトリに書かれたものだけが真実
- **構造的制約** — linter、型システム、アーキテクチャルールで「そもそも間違ったコードが書けない」状態を作る。エラーメッセージに修正手順を埋め込めば、エージェントが自己修正できる
- **自動ゴミ収集** — エージェント生成コードは人間と異なる形で劣化する（重複、未使用コード、型の劣化）。定期的に品質スキャンし、リファクタリングPRを自動生成する

なぜハーネスがモデルより重要なのか

モデルの性能は開発者がコントロールできない。次のバージョンが出れば変わるし、プロバイダの判断で挙動が変わることもある。一方、ハーネスは開発者が完全にコントロールできる。linterのルール、テストの網羅性、型の厳密さはすべて自分たちで設計できる。

さらに、ハーネスは**複利で効く**。一度追加したlinterルールは、以後のすべてのコード生成に適用される。テストは書いた瞬間から永続的に品質を検証し続ける。モデルの改善は一時的だが、ハーネスの改善は蓄積する。



ハーネスの中核はLint・テスト・型システム

人間がコードを書いていた時代、lintやテストは「品質向上の手段」だった。エージェントがコードを書く時代、それらは「品質の唯一の防衛線」になる。エージェントの出力が正しいかどうかを判定できるのは、機械的な検証だけ。OpenAIはAIスロップの手動清掃に多大な時間を費やしていたが、linterとテストの自動化で解消した（Harness Engineering, 2026）。

ハーネスの階層

層	役割	例
型システム	コンパイル時に不正を排除	TypeScript strict, Rust borrow checker
Lint	スタイルとパターンの強制	ESLint, clippy, ruff
テスト	振る舞いの検証	ユニット, 統合, E2E
CI/CD	デプロイ前の最終ゲート	GitHub Actions, Cloud Build

再発防止の仕組み化

エージェントがミスをしたら:

1. そのミスを検知するlinterルールを追加する
2. rulesにNEVER制約を追加する
3. hooksで自動ブロックする

人間への「気をつけてね」ではなく、機械的にブロックする。**信頼性は規律からではなく自動化から生まれる** — これはSREの品質管理プロセスとまったく同じ構造。



リポジトリの発酵と腐敗

人間がリポジトリを読むとき、「このREADMEは古そうだな」と直感的に判断できる。エージェントにはこの判断ができない。3ヶ月前の設計メモも昨日のコミットも、コンテキストに入れば同じ重みで扱われる。つまり、リポジトリに古い情報が残っているだけで、エージェントの出力品質が下がる。

リポジトリ内の情報は2種類に分かれる。**時間が経つほど信頼性が高まるもの**と、**時間が経つほど害になるもの**。テストは前者の典型。実行すれば正しさが検証されるし、コードが変われば即座に失敗して更新を迫る。linter設定やADR (Architecture Decision Records、ステータスとタイムスタンプ付き) も同様に「実行や構造で鮮度が担保される」情報。

一方、自然言語で書かれた説明文書、システム状態の記述（「現在のDB構成は～」）、スタイルガイドは後者。コードの進化に追いつけず、放置すればエージェントが古い前提で誤ったコードを生成する。古いTODOコメントも危険で、エージェントが「未実装だ」と判断して不要なコードを足すことがある。

古い情報からエージェントを守る

対策は2つある。エージェントから見えない場所に移すか、情報の鮮度を自動で維持する仕組みを作るか。

エージェントの視界から外す

CLAUDE.md内のHTMLコメント（`<!-- -->`）はv2.1.72から自動注入時に非表示になった。人間がメモを残しつつ、エージェントには見せない使い方ができる。

古い設計文書はWikiや外部ツールに移し、CLAUDE.mdにはリンクだけ残す。リンク先が消えればエラーで気づくが、古い文書がリポジトリの奥に眠っていても誰も気づかない。見えないことが最大の防御になる。

鮮度を自動で維持する

仕様の説明を書く代わりにテストを書く。テストが通っている限り仕様は正しい。設計判断はADRに記録し、ステータス（Accepted / Superseded / Deprecated）を必ず付ける。エージェントはSupersededマークを見て新しい判断を参照できる。

CIにデッドリンクチェックと未使用ファイル検出を入れる。さらに進めると、エージェント自身にリポジトリを定期走査させて、古いドキュメントや未使用コードのリファクタリングPRを自動生成させることもできる。

実行で検証できない情報は、時間とともにエージェントの敵になる。



ハーネスを構成するもう一つの柱 テスト戦略

Lint・型チェックでコードの形を守り、テストで振る舞いを検証する

E2Eテストはもはや省略できない

10年前は「E2Eテストなんてデモ前に手動確認すれば十分」と思っていた。今の自分から言わせれば、それは個人の能力を過信している。OAuth2フローのE2Eテスト — 複数リダイレクト、Cookie管理、セッション状態の確認 — を毎回手動で確認するのは人間の注意力の限界を超えている。「疲れていたから見落とした」で本番障害が起きるのは、個人の問題ではなく構造的な失敗。

エージェント時代に必須になった理由

エージェントが生成したUIコードを人間が目視確認するのは現実的ではない。Playwright MCPやPlaywright CLIを使えば、エージェント自身がブラウザを操作してE2Eテストを実行し、結果を検証できる。**生成→検証→修正のループが人間を介さずに自動で回る。**

ツール選択の指針

ツール	特徴
Playwright MCP	MCP経由でブラウザ操作。コンテキスト消費大
Playwright CLI	シェルコマンド経由。MCPより軽量
agent-browser (Vercel)	ref方式でセレクト脆性を排除。最高効率
Hurl	HTTP APIテスト。宣言的で軽量

「疲れていたから見落とした」は個人の問題ではなく、手動検証に依存する設計の問題。



E2Eテストのツール選定と実装戦略

実際にE2Eテストをエージェントのワークフローに組み込もうとすると、ツール選定で迷う。ポイントは2つ。**トークン効率**（MCP ツール定義のコンテキスト消費）と**セクタの安定性**（UIが変わってもテストが壊れないか）。

対象ごとに何を使うか

対象	推奨ツール
Web	Playwright CLI、agent-browser（Vercel）
Mobile	XcodeBuildMCP（iOS）、mobile-mcp、Detox（React Native）
API	Hurl（宣言的HTTP、軽量）、Pact（契約テスト）
CLI	bats-core、pexpect（対話操作）
Desktop	Playwright Electron、tauri-driver
インフラ	<code>terraform test</code> + Conftest（OPA）

Web E2Eの3つの選択肢

Playwright MCPはMCPプロトコル経由でブラウザを操作する。直感的だが、ツール定義がコンテキストを消費する。
Playwright CLIはシェルコマンド経由で同じことができ、トークン効率が高い。

agent-browser（Vercel Labs）はDOM要素に一意的IDを自動付与する方式で、CSSセクタへの依存がない。UIが変わってもIDは再付与されるので、テストの頑健性が高い。

いずれもエージェントが自分でブラウザを操作→結果を検証→問題があれば修正、というループを自動で回せる。



AI時代にテスト戦略は根本から変わる

2025年はAIの速度の年、2026年はAIの品質の年。DORA 2025のデータが示す現実: AI導入率95%、PR数+98%/人、しかしバグ率+9%、レビュー時間+91%。個人の生産性は上がったが、組織のデリバリー指標は横ばい。テストがこのギャップを埋める。

テストの役割が変わった

- **仕様書** — テストが「何を作るか」を定義する。プロンプトより曖昧さが無い
- **生きたドキュメント** — 実行すれば嘘をつけない。説明文書は腐敗する
- **人間とエージェントの契約** — 「完了」の定義をテストが機械的に判定する
- **品質のガードレール** — エージェントが間違ったとき、テストが教える

AI生成コードのエラー率

CodeRabbitの470PR分析:

- AI生成PRの 이슈数: **人間の1.7倍**
- ロジックエラー: **1.75倍**
- 可読性の問題: **3倍以上**
- セキュリティ問題: **最大2.74倍**
- パフォーマンス退行: **約8倍**

これらを人間のレビューだけで捕捉するのは不可能。テストが唯一のスケラブルな検証手段。



個人と組織の生産性ギャップを埋めるのはテストだけ。

TDDはエージェント時代にこそ効く

DORA Report 2025: 「AIは増幅器であり、TDDのような優れたプラクティスをさらに効果的にする」。テストを先に書くことで、エージェントが「壊れた実装を検証するテスト」を書くチートを防ぐ。テストが先にあれば、エージェントはそのテストを通すことに集中する。

なぜTDDがエージェントに効くのか

エージェントはプロンプトより**テストの方が仕様として正確に理解する**。「ユーザー登録時にメールを送る」という自然言語より、`test_registration_sends_email()`の方が曖昧さがない。

ただし注意点がある。TDAD論文（2026年3月）によると、TDDの手順を冗長に説明するとコンテキストを圧迫して逆効果になる。**テストコード自体を渡す方が、テスト方法論の説明より効く**。

プロパティベーステストの可能性

Anthropicの研究で、Claudeがプロパティベーステストを使って100以上のPythonパッケージから**984件の潜在バグを発見**。手動レビューで56%が実際のバグ、32%がメンテナに報告する価値あり。NumPy、AWS Lambda Powertoolsにパッチがマージされた。

エッジケースを人間が網羅的に考える必要がない。プロパティ（不変条件）を定義すれば、エージェントが自動でテストケースを生成する。



テストのないプロンプトは、正しさの定義がない依頼。

フィードバック速度が品質を決める

エージェントにとってフィードバックの速度は品質に直結する。CIが10分後に「失敗」と返すのと、ファイル保存直後にlinterが「ここが違う」と返すのでは、修正の精度がまったく異なる。

コンテキストは流れ去る

エージェントはコンテキスト内の情報で自己修正する。フィードバックが返る前に新しい作業が進むと、修正時の文脈が変わっている。10分後のCI失敗は「10分前の文脈」の修正を要求するが、エージェントは既に先に進んでいる。

速いフィードバックは自律を可能にする

速いフィードバックはエージェントの自己修正ループを回す。遅いフィードバックは人間の介入を待つ。つまりフィードバック速度は、どこまでエージェントに任せられるかの上限を決める。速い層ほど自律度を上げられる。

フィードバックが遅いほど、文脈が流れ去り、手戻りが膨らむ。



フィードバックの階層設計

フィードバックの速度は一律ではない。検証の種類ごとに最適なタイミングがあり、それぞれ異なるメカニズムで実現する。速い層ほど機械に任せ、遅い層ほど人間の判断を活かす。

4つの層

層	速度	検証内容
PostToolUse	ミリ秒	format, lint
プリコミット	秒	型チェック, テスト
CI/CD	分	統合テスト, E2E
人間	時間～	設計判断, 要件適合

上の層ほどエージェントが自律的に修正でき、下の層ほど人間の関与が必要になる。

「速度を上げるとオーバーヘッドが増える」への反論

PostToolUseフックは同期実行するとエージェントの操作をブロックする。しかしformat程度は100ms以内で終わる。lintが重い場合は `async: true` にしてバックグラウンド実行する。

速度のコストは設計で制御できるが、遅いフィードバックのコスト（手戻り、人間の介入）は制御できない。制御可能なコストと制御不能なコストなら、制御可能な方を選ぶ。

速い層ほど機械に、遅い層ほど人間に。



エラーメッセージを修正指示にする

普通のlinterは「ここが違反」と指摘するだけ。エージェント向けには、エラーメッセージそのものに**何をどう直すかの手順**を含める。PostToolUseフックの返却に含まれるので、エージェントは次のアクションで即座に修正を試みる。

従来のエラーメッセージ vs 修正指示

```
ERROR: ServiceAはInfraレイヤーに  
直接依存できません  
WHY: ADR-007参照  
FIX: Providerインターフェースを  
Domain層に定義してください
```

「何が間違っているか」だけでなく「なぜ間違っているか」「どう直すか」まで含める。エージェントがADRを参照して設計判断の文脈を理解し、正しい修正を選択できる。

構造的な効果

修正手順付きのエラーメッセージは、人間にとっても有益。新メンバーが初めてプロジェクトのアーキテクチャルールに触れたとき、「なぜこう設計されているか」がエラーメッセージから学べる。エージェントのためのインフラが、同時に人間のためのドキュメントとして機能する。

ここに**ハーネスの複利効果**がある。一度作ったエラーメッセージテンプレートは、以後のすべてのエージェントセッションと、すべての新メンバーのオンボーディングに効く。

エージェントのためのインフラは、人間のためのインフラでもある。



生成速度と理解速度の非対称

ここまでハーネスの各要素 — Lint、テスト、E2E、フィードバック速度 — を見てきた。ここからは、これらが積み重なった先にある構造的な変化を考える。

生成は並列化できるが、理解は並列化できない

エージェントを複数走らせれば生成速度は上がる。しかし人間の理解速度は並列化できない。コードベースの全体像を把握するのは個人の認知作業であり、人を増やしても線形には速くならない。生成のスケール曲線と理解のスケール曲線は根本的に形が違う。

理解されないコードは複利で負債になる

エージェントが数時間で書いた機能を、チームが何年も運用する。理解されていないコードが1万行あると、次の変更の影響範囲が推論できない。結果として「触らない方が安全」な領域が広がり、システム全体の変更容易性が劣化する。これは技術的負債とは別種の**理解の負債**。

生成は並列化できるが、理解は並列化できない。



「書いた人が直す」前提の崩壊

従来の開発・運用は「コードを書いた人間がいる」前提で成り立っていた。設計意図を知る人間がレビューし、障害時にはその人間が原因を推論し、修正する。エージェントがコードを書く時代に、この前提が崩れる。

障害時に誰が直すのか

エージェントが生成したコードで障害が起きたとき、そのコードの設計意図を知る人間がいない。「速く書けた」は「速く直せる」と同義ではない。障害時の推論時間が延び、MTTRが悪化する。

ハーネスが前提を置き換える

人間が全コードを理解していなくても、テスト・linter・型チェックが通っていれば影響範囲を特定できる。ADRで判断の履歴を残していれば、エージェント自身に障害調査を依頼することもできる。人間の理解に依存しない運用設計が必要になる。

誰も理解していないコードが本番で動く。その前提で設計せよ。



レビュワーはもう育てられない

品質を維持したままコードレビューを軽量化し続ける必要がある。レビューを0にするという話ではない。ただ、今のままの人間中心のレビューには構造的な限界がある。最初の限界は、レビュワーの再生産が止まること。

育成方法が消える

コードレビュー能力の育成方法として、自分でコードを書いて経験を積む以外の道はまだ確立されていない。エージェントがコードを書く時代に、現在のレビュワーと同程度にコードを書く時間を確保することは現実的に難しい。レビュー能力の再生産が構造的に止まる。

「AIが書いたコードを読む力は別では」への反論

「エージェントが書いたコードを読む能力を鍛えればいい」という反論がある。しかし「読む力」も「書く経験」から来る。書いたことのないパターンのコードを的確にレビューできる人間は稀。さらに生成量が増えれば、読む対象自体が人間のキャパシティを超える。

書く機会が減れば、レビュー能力の再生産は止まる。



生成量がレビュー容量を超える

2つ目の構造的な限界。エージェントの生成速度は今後さらに上がるが、人間のレビュー速度は上がらない。DORA 2025のデータ: PR数+98%/人、レビュー時間+91%。生成がレビューのキャパシティを超えている現実がすでにある。

品質のばらつきが許容できない

人間のレビューは疲労、注意力、経験に左右される。朝一番のレビューと金曜夕方のレビューで品質が同じ保証はない。100件のPRのうち1件のレビュー漏れが本番障害を起こす。機械的な検証にはこのばらつきがない。

「レビューを減らすと品質が下がる」は本当か

直感に反するが、ハーネスがレビューの検証項目を機械的にカバーできるなら、レビューを軽量化した方がむしろ品質は安定する。人間のレビューは品質にばらつきが大きすぎる。機械的な検証にはばらつきはない。品質を保ったまま軽量化に成功した組織は、そうでない組織よりデリバリーが速い。

人間のレビューは疲れる。機械の検証は疲れない。



ではレビューをどう再設計するか

問題は明確になった。レビューワーカーの再生産は止まり、生成量はレビュー容量を超える。ではレビューを捨てるのか。そうではない。レビューの「何を」「誰が」を再定義する。

レビューで見えていたものを分解する

コードレビューで人間が見ていたものは5つに分解できる。フォーマット、命名規則、セキュリティパターン、ロジックの正しさ、設計の妥当性。前3つはlinter・静的解析で自動化できる。ロジックはテストで検証できる。人間にしかできないのは**設計の妥当性の判断**だけ。つまりレビューの8割は機械に移管可能。

「減らす」ではなく「再配置」

レビューを減らすのではなく、各検証を最も適した担い手に再配置する。フォーマットはlinterに。ロジックはテストに。セキュリティは静的解析に。人間には設計判断だけが残る。総量は変わらない。担い手が変わる。

レビューを減らすのではなく、担い手を再配置する。



軽量なレビューで品質を保てる組織が生き残る

「レビューを減らす＝品質が下がる」は、レビューが品質保証の唯一の手段だった時代の前提。ハーネスが検証項目を機械的にカバーできるなら、この前提は崩れる。

競争原理が淘汰を起こす

ハーネスで品質を担保しながらレビューを軽量化した組織は、同じ品質でデリバリーが速い。これは「悪貨が良貨を駆逐する」ではなく、機械的な検証の方が人間のレビューより均質で信頼性が高いという構造的な優位。重いレビュープロセスに固執する組織は、軽量化に成功した組織に淘汰される。

ただし、ハーネスなしの軽量化は崩壊する

レビューだけを減らしてハーネスを整備しなければ、品質は確実に下がる。「レビュー軽量化＝品質低下」が成り立つのは、代替手段なしに省略した場合。ハーネスの整備が先、軽量化は後。この順番を守れない組織は、速度も品質も失う。

ハーネスの整備が先、軽量化は後。順番を間違えると崩壊する。



エージェントにセルフレビューを委譲する

「レビューを機械に」と言うと抽象的に聞こえるが、実際に自分の開発フローに組み込んでいる仕組みがある。コードを変更したら、コミット前にエージェント自身に自分のコードをレビューさせ、問題を自動修正させるセルフレビューのパターンだ。

複数の視点を並列で回す

1台のレビュアーでは視野が偏る。品質・セキュリティ・パフォーマンスを見るレビュアー、外部モデルの視点を入れるレビュアー、可読性と過剰な複雑さを検出するレビュアー。この3つを並列で走らせると、それぞれが異なる角度から指摘を出す。複数のレビュアーが同じ箇所を指摘したら、それは本当に直すべき問題。

指摘 → 評価 → 修正のループ

レビューが出たら、次のステップが重要。指摘を鵜呑みにせず、各コメントの技術的妥当性を評価するフェーズを挟む。「要件で意図的にそう設計した」「トレードオフとして許容済み」なら対応しない。対応しない理由も明示する。実際に10件の指摘から6件だけ採択し、4件を根拠付きで却下する — これが健全なレビューの姿。

レビュー指摘を鵜呑みにするエージェントは、要件を壊す。



セルフレビューが変えるもの

セルフレビューの仕組みは、人間のレビューを不要にするためではなく、人間がdiffを開く前に品質の底を引き上げるためにある。

「毎回ツッコミを入れる」をなくす

エージェントの出力は品質が安定しない。変数名、エラーハンドリング、不要なコメント – 同じ種類の問題を毎回人間が指摘するのは時間の無駄。セルフレビューで先に潰せば、人間がdiffを開いたときに目にするのは設計上の判断だけになる。

レビューの本来の目的を取り戻す

コードレビューの目的は「コードベースの健康状態を改善すること」のはず。しかし実態は変数名やフォーマットの指摘に時間を取られ、設計の議論に辿り着かない。些末な指摘を機械に委譲すれば、人間はコードベース全体の健康に集中できる。

品質の底上げを自動化し、人間は設計判断に集中する。



今のエージェントの限界と、それでも作る理由

正直に言えば、今のエージェントは全体感のない実装をしがちだ。既存コードとの一貫性を無視したり、プロジェクト全体の設計方針から外れた局所最適な変更を出してくる。セルフレビューの指摘も、今は変数名や未使用インポートのような表層的なものが多い。

全体感の欠如は構造的な限界ではない

コンテキストウィンドウは1Mトークンに拡大し、モデルの推論能力は四半期ごとに向上している。全体感の欠如は「今の」エージェントの一時的な限界であって、アーキテクチャの壁ではない。セルフレビューの仕組みを今作っておけば、モデルが進化するたびに自動的にレビューの質も上がる。

ハーネスは今作る。モデルは後から来る

セルフレビューの仕組み — 複数エージェント並列、指摘の批判的評価、修正ループ — これはモデルに依存しない構造。モデルが賢くなれば、同じ仕組みでより深い指摘が出る。粗い指摘しか出ない今だからこそ、仕組みを先に作る意味がある。仕組みは複利で効く。

仕組みは今作る。モデルの進化が後から仕組みを強くする。



品質保証の3つの柱

コードレビューを軽量化するには、レビューで担保していた品質を別の手段に置き換える必要がある。

1. 高品質なコンテキスト

エージェントが高品質なコードを生成するための入力を整備する。CLAUDE.md、rules、ADR、テスト。これらのコンテキスト自体の開発・維持もエージェントが支援できる。生成AIの文書化能力は多くの人間を超えており、均質なコンテキストを維持するコストは下がり続ける。

2. 高品質なハーネス

linter、静的解析、型チェック、テストスイート。これらがコードレビューで人間が見ていた部分を機械的に置き換える。プロダクト固有のアーキテクチャやフレームワーク自体もハーネスの一部。コードの形を制約すれば、レビューで確認すべき範囲が狭まる。

3. 人間の評価を上位レイヤーに集中させる

コンテキストとハーネスがコード品質を担保した上で、人間はコード行ではなく「要求への適合」を評価する。フォーマット、命名、セキュリティパターンは機械に任せ、設計の妥当性と要件の整合だけが人間の仕事として残る。

入力をコンテキストで、出力をハーネスで、構造を人間で。



人間の評価はコードからより上位のレイヤーへ

コンテキストとハーネスがコード品質を担保する。では、人間は何を評価するのか。コード行レベルのレビューから、要求・要件への適合度の評価に移行する。

評価システム自体もエージェントが生成する

コンテキスト（要求仕様、ADR、テスト）をインプットにして、コード生成とは別プロセスで評価基準を自動生成できる。言語化された領域の評価は、近いうちに人間よりエージェントの方が高い精度でできるようになる。人間が最終的に評価するのは、最上位要件への適合と、まだ言語化されていない領域。

レビューは消えない、形が変わる

人間がコードを目視する機会は減る。しかし、レビューという考え方自体がプロセスから消えるわけではない。linterや静的解析のような自動化された品質保証プロセスが、人間がレビューしていた部分を置き換えていく。人間の役割は、コード行の精査から、**構造と要件の整合を検証する仕事**に変わる。

コードを読める人は減る。問いを立てられる人の価値は上がる。



「ジュニアの育成にレビューが必要」という反論

「レビューを減らすとジュニアが育たない」は最もよく聞く反論。これは部分的に正しい。ただし、レビューの教育機能と品質保証機能を混同している。

品質保証と教育は別のプロセス

品質保証は「このコードを本番に出して良いか」の判定。教育は「このエンジニアの設計判断力を育てる」プロセス。同じレビューで両方やっていたのは、人間がボトルネックだった時代の最適化でしかない。品質保証を機械に移せば、教育のための時間が生まれる。

教育手段はレビュー以外にもある

ペアプログラミング、設計ドキュメントのレビュー（コードではなく設計意図）、ADRの共同作成、モブプログラミング。むしろdiffベースのコードレビューは、教育手段としては受動的すぎる。「なぜこの設計にしたか」を議論するの方が学びが深い。

品質保証と育成を混同すると、両方が中途半端になる。



まとめ

この発表で見てきたことを振り返る

3つの持ち帰り

1. 道を示してから走らせる

PLAN MODEで計画を立て、人間がレビューし、承認してから実装に移る。計画のアノテーション（Phase 2.75）がエージェントの理解の限界を可視化する。制約のないエージェントは「自分なりの最善」を実行する — それはプロジェクトの最善ではない。

2. ハーネスを構築する

CLAUDE.mdは索引。rulesで関心事を分離。hooksで品質チェックを自動化。テストで振る舞いを検証。linterでスタイルを強制。ハーネスの違いがモデルの違いより圧倒的に性能に効く（Can Boluk, 2026）。投資すべきはハーネス。

3. 知り、整理し、更新し続ける

3ヶ月で80リリース。新機能を知り、自分のrules/skills/hooksに反映し、古くなった情報を刈り取る。この継続的な再整理がチームのハーネスを育てる。個人のツールで終わらせず、リポジトリにコミットしてチームの形式知にする。

正しいコードを書くのではなく、正しいコードが生まれる構造を作る。



参考資料（1）公式ドキュメント・自著ブログ

Anthropic公式

- [Claude Code Overview](#) — 公式ドキュメントハブ
- [Best Practices for Claude Code](#) — 公式ベストプラクティス
- [Extend Claude with Skills](#) — スキル公式ドキュメント
- [Claude Code Hooks Reference](#) — フック公式リファレンス
- [How Anthropic Teams Use Claude Code](#) — 社内利用パターン
- [Effective Harnesses for Long-Running Agents](#) — Anthropic Engineering（2025年11月）

自著ブログ（じゃあ、おうちで学べる）

- [Claude CodeのPLAN MODEは使ったほうがいい](#)（2026年2月）
- [SREはAgentic Engineering時代のHarnessになれるのか？](#)（2026年3月）
- [Claude Codeの Agent Skills は設定したほうがいい](#)（2025年12月）
- [Claude CodeのSubagentsは設定したほうがいい](#)（2025年9月）
- [Claude Code の CLAUDE.mdは設定した方がいい](#)（2025年6月）



参考資料 (2) ハーネスエンジニアリング

原典・一次ソース

- [Harness Engineering: Leveraging Codex in an Agent-First World](#) — Ryan Lopopolo, OpenAI (2026年2月)
- [My AI Adoption Journey](#) — Mitchell Hashimoto (2026年2月) 「エージェントがミスをするたびに、二度とそのミスを起こせない仕組みを作る」
- [Harness Engineering](#) — Birgitta Böckeler, Thoughtworks / Martin Fowler (2026年2月)
- [The Emerging Harness Engineering Playbook](#) — Charlie Guo, OpenAI (2026年2月)
- [Skill Issue: Harness Engineering for Coding Agents](#) — HumanLayer (2026年3月)
- [I Improved 15 LLMs at Coding in One Afternoon](#) — Can Boluk (2026年2月)

コミュニティ

- [everything-claude-code](#) — エージェントハーネス性能システム
- [Writing a Good CLAUDE.md](#) — HumanLayer



参考資料 (3) セキュリティ・その他

OWASP・セキュリティ

- [OWASP Top 10 for Agentic Applications \(2026\)](#) — OWASP GenAI Security Project (2025年12月)
- [ToxicSkills: Malicious AI Agent Skills](#) — Snyk (2025年) 公開スキルの36%にプロンプトインジェクション
- [IDESaster: 30+ Vulnerabilities in AI IDEs](#) — Ari Marzouk (2025年12月) 24 CVEs
- [Claude Code Security](#) — Anthropic (2026年2月)
- [Lessons from OWASP Top 10 for Agentic Applications](#) — Auth0

評価・品質

- [Eval-Driven Development](#) — EDD Manifesto
- [Improving Deep Agents with Harness Engineering](#) — LangChain
- [Building Effective AI Coding Agents for the Terminal](#) — arXiv (2026年3月)

関連ツール

- [Oxlint 1.0](#) — ESLint比50-100倍高速、Rust製linter
- [Biome v2.0](#) — フォーマッター+linter統合
- [Lefthook](#) — Git hooksマネージャー
- [Hurl](#) — HTTP APIテスト (トークン効率最高)





3>SHAKE

ありがとうございました

@nwiizo

